Business Loans brand ranking — from research scripts to a production system

Elky · MLOps assignment, Schemathics · repo: github.com/elkyss/schemathics-mlops

> **In one line:** the researcher's two scripts run **unchanged** inside a weekly, MLflow-tracked training pipeline with a gated registry and one-command rollback, and behind an API that answers in ~15 ms of our own code plus the payout model. The biggest win came from the TabPFN finding: the original predictor took **8.9 s per user** because it re-fits TabPFN through the hosted API on every request. Fitting once and reusing the persisted fit brings this to **1.7 s** (hosted) or **0.28 s** (local), with a fallback that answers within the deadline.

1 · WHAT I BUILT

- **Untouched model logic.** The originals are vendored byte-identical (sha256 test against the upstream commit). Thin subclasses change plumbing only; TabPFN backends are swapped by rebinding the module's `TabPFNRegressor` name. A train-serve parity test proves the API returns exactly what the researcher's predictor returns.
- **Data as a table.** CSV → DuckDB table (Delta on Databricks), all columns text. Every run snapshots it to a CSV that `pd.read_csv` reads identically to the source (zero-padded `sub1`, 18-digit IDs), logs its sha256 and validates it before training.
- **Training pipeline** (`bl train`): both modes; per-day classification and per-threshold payout metrics in MLflow; the researcher's log as an artifact; library versions, git and upstream commit, row counts per filter. Production registers `bl_ranker` as one pyfunc (CatBoost + TabPFN context + persisted fit id + brand list + the original preprocessing) with a model card.
- **Lifecycle:** `@challenger` → gate → `@champion`; every promotion records the version it replaced, so rollback is one alias move; the API hot-reloads on `/admin/reload` (no restart).
- **Serving:** FastAPI loads the champion once. The response body is the original dict; version, backend, degraded and cache flags travel in headers. Prometheus `/metrics`, JSON logs, and an inference log without personal data.
- **Runs anywhere:** `make demo` runs the whole story in Docker (train → serve → retrain with gate → rollback), verified from a fresh clone. `ENV=databricks` + an asset bundle maps the same code to UC tables, Databricks MLflow, Jobs cron and Model Serving. CI runs lint, 89 tests and the image build.

2 · DILEMMAS AND DECISIONS

How to "not change the logic" and still fix it? I wrapped rather than edited: overriding a method and calling `super()` keeps the researcher's code the single source of truth, so their next version drops in. Where the original is wrong I fixed around it and documented it: production mode never trained on the last 7 days (the split always runs, leaving the TabPFN context a week stale), so production now passes 0 holdout days (a flag restores the original). The predictor reads a *global* `user_data`, which I set under a lock.

Hosted vs local TabPFN was the key trade-off, and my first measurement was wrong: repeated rows hit a server cache (0.9 s). Real users always send new rows:

Payout backend	Latency, new rows	Payout MAPE	Cost / licence
Hosted API (researcher's choice; `TABPFN_BACKEND=api`)	1.5–1.8 s	5.4 %	~10k credits per call; free tier ≈ 2,000/month
Local OSS TabPFN-3.5, CPU, cached context (**default**)	0.21 s	8.9 %	free; ~5.5 GB RAM/process; weights non-commercial
Fallback: mean payout per brand	0.015 s	—	safety net only

Both are configuration (`TABPFN_BACKEND`, or `SERVE_PAYOUT_BACKEND` at serve time, no retraining). The **default is local**: ~7× faster, no per-call cost or quota, no data egress. The hosted model is more accurate, so choosing between them is a business call for research. Production on local weights needs Prior Labs' commercial licence. Training persists the hosted fit's id (fit is free, only predicts cost), so serving never re-uploads. A 2 s deadline sends slow calls to the fallback; one warm-up call per model load removes the cold first request.

A fair promotion gate. Comparing a new model with the champion's *old* metrics compares different weeks. The gate scores both on the same newest week, and runs only if the champion never trained on that week. Otherwise the new version waits at `@challenger` for a human; an unchanged snapshot registers nothing.

What to build vs. describe. In 48 h I prioritised what a reviewer can run: Docker, fresh-clone check, tests. I described rather than built Kubernetes, Postgres/MinIO, feature stores and A/B infrastructure.

3 · HOW IT RUNS IN PRODUCTION

- **Sunday 05:00** (Israel time; DST-safe): snapshot → validate → gate → train on all data → register → promote if the gate passes (or wait for approval with `AUTO_PROMOTE=false`) → API hot-reload. Failure = non-zero exit, FAILED MLflow run, stale `last_success`; single-run lock.
- **Per request:** validate → original preprocessing for ~10 brands → CatBoost P(lead) × payout → rank; cached 10 min per request+version; degraded answers are never cached.
- **Watch:** p50/p99, error rate, fallback rate, cache hits, model version and age (`/metrics`); gate decisions and model cards in the registry; the inference log for drift and outcomes.
- **Incidents:** bad model → `bl rollback` (seconds). TabPFN API down → automatic fallback per request, or `SERVE_PAYOUT_BACKEND=local`.

4 · MEASURED RESULTS

Scenario (MacBook M1 Pro, 1 process)	p50	p99	Note
Original predictor, per request (hosted TabPFN)	8.9 s	33.6 s max	+ crashes on ~5 % of users
Original predictor, per request (local TabPFN)	5.0 s	5.0 s	2 of 20 crashed
Ours · hosted API · realistic (n=40, quota-capped)	1.7 s	2.1 s	25 % answered by fallback
Ours · local · realistic (5 users)	0.28 s	0.76 s	SLO ≤ 0.4 s ✘ under concurrency
Ours · fallback · realistic / stress	36 ms	84 ms	~64 rps per process, cache off

Production load is ~2,000 registered sessions/day (≈0.02 rps), so one small replica covers peak. Replicas are for availability. Each replica needs ~2.3 GB, mostly NameDataset.

5 · ISSUES FOUND AND OPEN RISKS

- **Crash:** users with < 5 of 8 survey answers (~5 %) make the original predictor crash. The API returns `{}` with a reason instead.
- **Hosted TabPFN in production:** latency, quota (~60k predictions/month needed vs 2,000 free), per-request **data egress** to a third party (needs a DPA), and eviction of server-side fits (handled by refit + fallback).
- **Fair lending:** the model uses **gender inferred from first names** in a lending product. That's an ECOA/Reg B risk to raise with research.
- **Model gaps:** the 1,000-row TabPFN context covers only 4 of 10 brands; `sub1` arrives as an int in the example request but as a zero-padded string in the data; day-of-month as a feature; test-week gaps (days with 3–23 rows make per-day metrics noisy).

6 · NEXT STEPS

- **Close the feedback loop:** join `inference_log` with later dispositions and payouts → realised **revenue per session per model version** (the real KPI), plus calibration of P(lead).
- **Roll out by evidence:** shadow-score the challenger, then a session-hash A/B on revenue/session with guardrails. Offline F1 does not prove business lift.
- **Drift monitoring** (PSI on survey bands, rank-1 brand share), circuit breaker for the API, Redis cache shared across replicas, Postgres/MinIO behind MLflow, per-version model cards with fairness slices.
- **TabPFN decision for research:** local with a commercial licence (fast, private) vs hosted (more accurate). A larger, brand-stratified context may close the accuracy gap.

7 · ALTERNATIVES CONSIDERED

- **Editing the scripts** (cleaner code, but breaks "don't change the logic" and forks the researcher's code). I chose to wrap them, with a parity test.
- **`mlflow models serve` / Databricks Model Serving only** (less code, but no control over warm loading, deadlines, caching or headers). The same pyfunc still deploys there.
- **Distilling TabPFN into a fast regressor** (fastest, but changes the model). That's for research to decide.
- **Airflow / Prefect** for scheduling (too heavy for one weekly job). I used a small, tested cron loop locally and Databricks Jobs there.
- **Postgres + MinIO, Kubernetes** (closer to production, but costly for a reviewer to run). Each is a connection-string or deployment change away.